Jenkins – Despliegue Continuo

Alumno: Yamil Motta

Asignatura:Proyecto Final

Ciclo: 2ºASIR

Curso: 25/26

Entrega Continua **3**

Generar clave ssh **3**

Jenkins 3

AWS 3

Vincular a Jenkins clave privada **4**

Prueba SSH final **4**

Jenkinsfile con CD **5**

SAST y DAST 5

Primera versión Jenkinsfile 5

Segunda versión 12

Tercera versión 16

Cuarta versión 17

Quinta versión 19

Sexta versión 21

Séptima versión 22

Octava versión 23

Novena versión 23

Décima versión 25

Undécima versión 26

Duodécima versión 28

Trigésima versión 29

Décimocuarta versión 31

Decimoquinta versión 32

Decimosexta versión 32

Decimonovena versión 33

Pipeline Success 34

Entrega Continua

Generar clave ssh

Jenkins

Genero clave ssh ed25519 con nombre “jenkins_aws2_deploy”, sin palabra de paso

```
yamil@g1server-cicd:~$ ssh-keygen -t ed25519 -f ~/.ssh/jenkins_aws2_deploy -N "" -C "jenkins@tfg-cicd"
Generating public/private ed25519 key pair.
Your identification has been saved in /home/yamil/.ssh/jenkins_aws2_deploy
Your public key has been saved in /home/yamil/.ssh/jenkins_aws2_deploy.pub
The key fingerprint is:
SHA256:nmtWQI58cCgwP5bn6wpgpbDy34t4HBuBhj3oM5gOFCo jenkins@tfg-cicd
The key's randomart image is:
+--[ED25519 256]--+
|  o.   .         |
| . o..o o        |
|o+.o=o.*         |
|EoB..+o +        |
|O= . ...S.       |
|**  o  o ..      |
|o =. +. o.       |
| . +++  o.       |
|  ..+.++.        |
+----[SHA256]-----+
yamil@g1server-cicd:~$
```

AWS

Agregamos a “deploy” como usuario para jenkins (con privilegios reducidos):

```
ubuntu@ip-10-1-0-85:~/.ssh$ sudo useradd -m -s /bin/bash deploy
```

Pego la clave pública generada en AWS2 /home/deploy/.ssh/authorized_keys

```
Search sessions, agents, files...
  GNU nano 7.2                          /home/deploy/.ssh/authorized_key *
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK7/zz45m+miKc71MYFmGumjxpAkEXsWM3LqLTPL2ylS jenkins@tfg-cicd
```

Le doy los permisos que necesite:

```
ubuntu@ip-10-1-0-85:~/.ssh$ sudo chown -R deploy:deploy /home/deploy/.ssh
ubuntu@ip-10-1-0-85:~/.ssh$ sudo chmod 700 /home/deploy/.ssh
ubuntu@ip-10-1-0-85:~/.ssh$ sudo chmod 600 /home/deploy/.ssh/authorized_keys
```

Verificamos:

```
ubuntu@ip-10-1-0-85:/home$ sudo cat ./deploy/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK7/zz45m+miKc71MYFmGumjxpAkEXsWM3LqLTPL2ylS jenkins@tfg-cicd
ubuntu@ip-10-1-0-85:/home$ sudo ls -l ./deploy/.ssh/authorized_keys
-rw-r--r-- 1 root root 98 May 16 22:45 ./deploy/.ssh/authorized_keys
ubuntu@ip-10-1-0-85:/home$
```

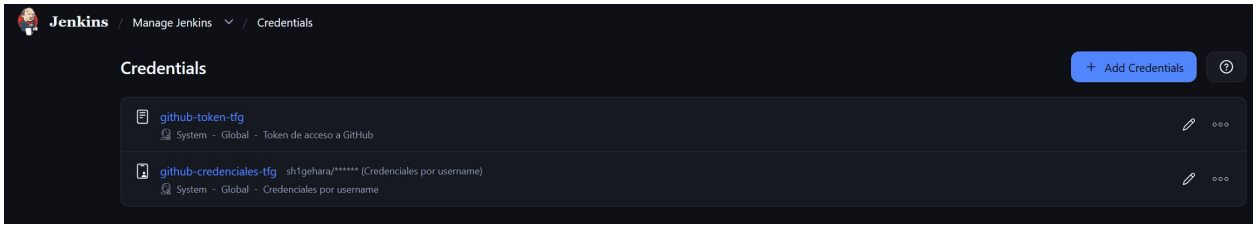
Agregamos al grupo de docker para que pueda tocar el contenedor Apache:

```
ubuntu@ip-10-1-0-85:/home$ sudo usermod -aG docker deploy
ubuntu@ip-10-1-0-85:/home$
```

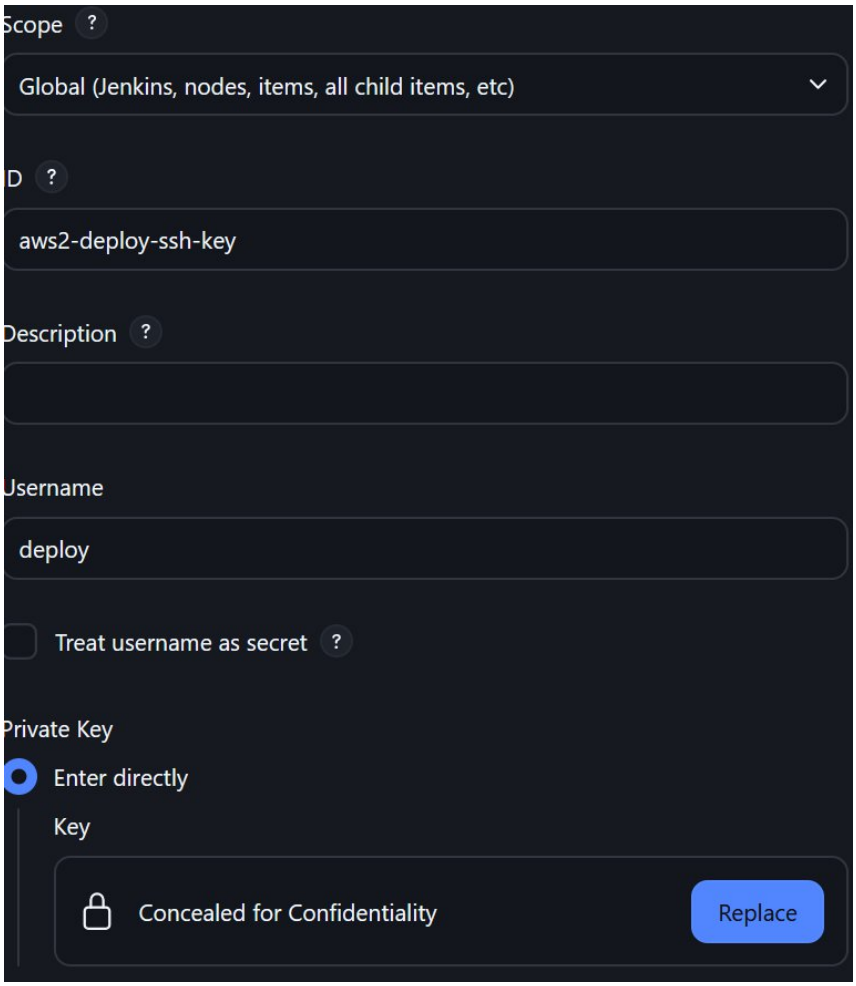
De esta manera, la clave pública de jenkins es reconocida por AWS.

Vincular a Jenkins clave privada

En Jenkins > Manage Jenkins > Credentials, vamos a crear una nueva credencial junto al token de github y las credenciales de mi usuario.



Añadimos unas credenciales del tipo SSH Username with private key



El nombre (ID) de la credencial quedará como aws2-deploy-ssh-key porque así quedará reflejado en el Jenkinsfile.

Prueba SSH final

Hacemos la prueba a conectar por ssh y ejecutar un comando docker a AWS2:

```
yamil@g1server-cicd:~/.ssh$ ssh -i ./jenkins_aws2_deploy deploy@10.100.0.3 'docker ps'
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS          PORTS
                         NAMES
a7c8c2050575   mi_proyecto_web_web    "docker-php-entrypoi…"   25 hours ago   Up 36 minutes   0.0.0.0:80-
>80/tcp, [::]:80->80/tcp   mi_proyecto_web_web_1
yamil@g1server-cicd:~/.ssh$
```

Jenkinsfile con CD

SAST y DAST

Con la conexión correctamente hecha, podemos avanzar con la pipeline. El CD contará con SAST y DAST:

1. **Análisis Estático - SAST:** En cuanto subimos una actualización del juego al repositorio, el sistema lee nuestro código fuente sin llegar a ejecutarlo. Semgrep y PHPStan funcionan como un corrector ortográfico avanzado que busca errores de programación en php, variables mal definidas o posibles brechas de seguridad evidentes en el texto del código.
2. **Análisis Dinámico - DAST:** Si el código está limpio, el sistema levanta una copia temporal y aislada de nuestro juego de damas junto con una base de datos de prueba. Sobre este entorno seguro entra en acción una herramienta automatizada (OWASP ZAP) que actúa como un atacante virtual. Su trabajo es interactuar con el juego real, creando partidas y mandando datos alterados para descubrir vulnerabilidades que solo dan la cara cuando la aplicación está funcionando. Al terminar, este entorno de prueba se destruye automáticamente.
3. **Despliegue:** Solo cuando la actualización ha demostrado ser segura y estable en las dos fases anteriores, el sistema se conecta por un túnel privado a nuestro servidor principal en la nube (AWS). Allí sustituye la versión antigua del juego por la nueva y reinicia los servicios, todo ello de forma automática y sin que tengamos que intervenir manualmente.

Primera versión Jenkinsfile

```
pipeline {
    agent none
    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '20'))
        disableConcurrentBuilds()
    }

    stages {
        stage('SAST: Lint sintáctico PHP') {
            agent { docker { image 'php:8.2-cli'; args '-u root' } }
            steps {
                sh 'php -l app/damas/api.php'
                sh 'php -l app/damas/GameLogic.php'
                sh 'php -l app/damas/DB.php'
            }
        }

        stage('SAST: PHPStan') {
            agent { docker { image 'ghcr.io/phpstan/phpstan:latest'; args '-u root --entrypoint="" } }
            steps {
                sh 'phpstan analyse app/damas --level=5 --no-progress --error-format=table || true'
            }
        }

        stage('SAST: Semgrep (Seguridad)') {
            agent { docker { image 'returntocorp/semgrep:latest'; args '-u root --entrypoint="" } }
            steps {
                sh 'semgrep --config=p/php --config=p/security-audit --error app/damas/ || true'
            }
        }

        stage('Levantar entorno efímero para DAST') {
            agent any
```

```
        steps {
            sh '''
                docker network create damas-test 2>/dev/null || true

                docker run -d --rm --name damas-db --network damas-test \
                    -e MYSQL_ROOT_PASSWORD=test \
                    -e MYSQL_DATABASE=damas \
                    -v $PWD/db/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro \
                    mysql:8.0

                for i in $(seq 1 30); do
                    docker exec damas-db mysqladmin ping -uroot -ptest --silent && break
                    sleep 2
                done

                docker build -t damas-web:test app/damas/
                docker run -d --rm --name damas-web --network damas-test \
                    -e DB_HOST=damas-db \
                    -e DB_USER=root \
                    -e DB_PASS=test \
                    -e DB_NAME=damas \
                    damas-web:test
                sleep 3
            '''
        }
    }

    stage('DAST: ZAP API Scan') {
        agent any
        steps {
            sh '''
                mkdir -p zap-reports

                # Interceptar variables de una partida real para el fuzzing
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                    -s -X POST -H "Content-Type: application/json" \
                    -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                    "http://damas-web/api.php?accion=nueva")

                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                # Inyectar variables en el OpenAPI
                sed -e "s/abc12345/$CODIGO/g" \
                    -e
"s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g" \
                    app/damas/openapi.yaml > zap-reports/openapi-runtime.yaml

                # Ejecutar escaneo activo
                docker run --rm --network damas-test \
                    -v $PWD/zap-reports:/zap/wrk:rw \
                    ghcr.io/zaproxy/zaproxy:stable \
                    zap-api-scan.py \
                        -t /zap/wrk/openapi-runtime.yaml \
                        -f openapi \
                        -r zap-api-report.html \
                        -J zap-api-report.json \
                        -I
            '''
        }
        post {
            always {
                archiveArtifacts artifacts: 'zap-reports/*', allowEmptyArchive: true
                publishHTML target: [
                    reportDir: 'zap-reports',
                    reportFiles: 'zap-api-report.html',
                    reportName: 'OWASP ZAP Report',
                    keepAll: true
                ]
```

```
            }
        }
    }

    stage('Desmontar entorno efímero') {
        agent any
        steps {
            sh '''
                docker rm -f damas-web damas-db 2>/dev/null || true
                docker network rm damas-test 2>/dev/null || true
            '''
        }
    }

    stage('Deploy a Producción (AWS2)') {
        when { branch 'main' }
        agent any
        steps {
            sshagent(['aws2-deploy-ssh-key']) {
                sh '''
                    rsync -avz --delete \
                        -e "ssh -o StrictHostKeyChecking=accept-new" \
                        app/damas/html/ \
                        ubuntu@10.100.0.3:/home/ubuntu/mi_proyecto_web/html/

                    ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 \
                        'cd /home/ubuntu/mi_proyecto_web && docker compose up -d --build'
                '''
            }
        }
    }
}

post {
    always {
        node('built-in') {
            sh '''
                docker rm -f damas-web damas-db 2>/dev/null || true
                docker network rm damas-test 2>/dev/null || true
            '''
        }
    }
    failure {
        echo 'Pipeline Fallida — El código no ha alcanzado AWS2.'
    }
    success {
        echo 'Pipeline OK — Despliegue completado.'
    }
}
}
```

Consola

El primer lanzamiento de esta pipeline con SAST y DAST no daba problema, hasta llegar a la base de datos efímera.

```
[Pipeline] sh
+ semgrep '--config=p/php' '--config=p/security-audit' --error app/damas/


┌──────────────┐
│ Scan Status │
└──────────────┘
  Scanning 7 files tracked by git with 249 Code rules:

  Language      Rules   Files          Origin      Rules
 ─────────────────────────────        ──────────────────
  <multilang>       9       4          Community     249
  php              23       3


┌─────────────────┐
│ Scan Summary │
└─────────────────┘
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 32
 • Targets scanned: 7
 • Parsed lines: ~100.0%
 • Scan was limited to files tracked by git
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 32 rules on 7 files: 0 findings.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
[Pipeline] }
$ docker stop --time=1 5ed45ccaabafbe95ab92f94d835b07a4ee40c67bd40226c4e3f355c275fc3ec3
$ docker rm -f --volumes 5ed45ccaabafbe95ab92f94d835b07a4ee40c67bd40226c4e3f355c275fc3ec3
[Pipeline] // withDockerContainer
```

```
+ docker run -d --rm --name damas-db --network damas-test -e MYSQL_ROOT_PASSWORD=test -e MYSQL_DATABASE=damas -v /var/jenkins_home/workspace/TFG-Pipeline-Prueba/db/
schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
```

Fallos

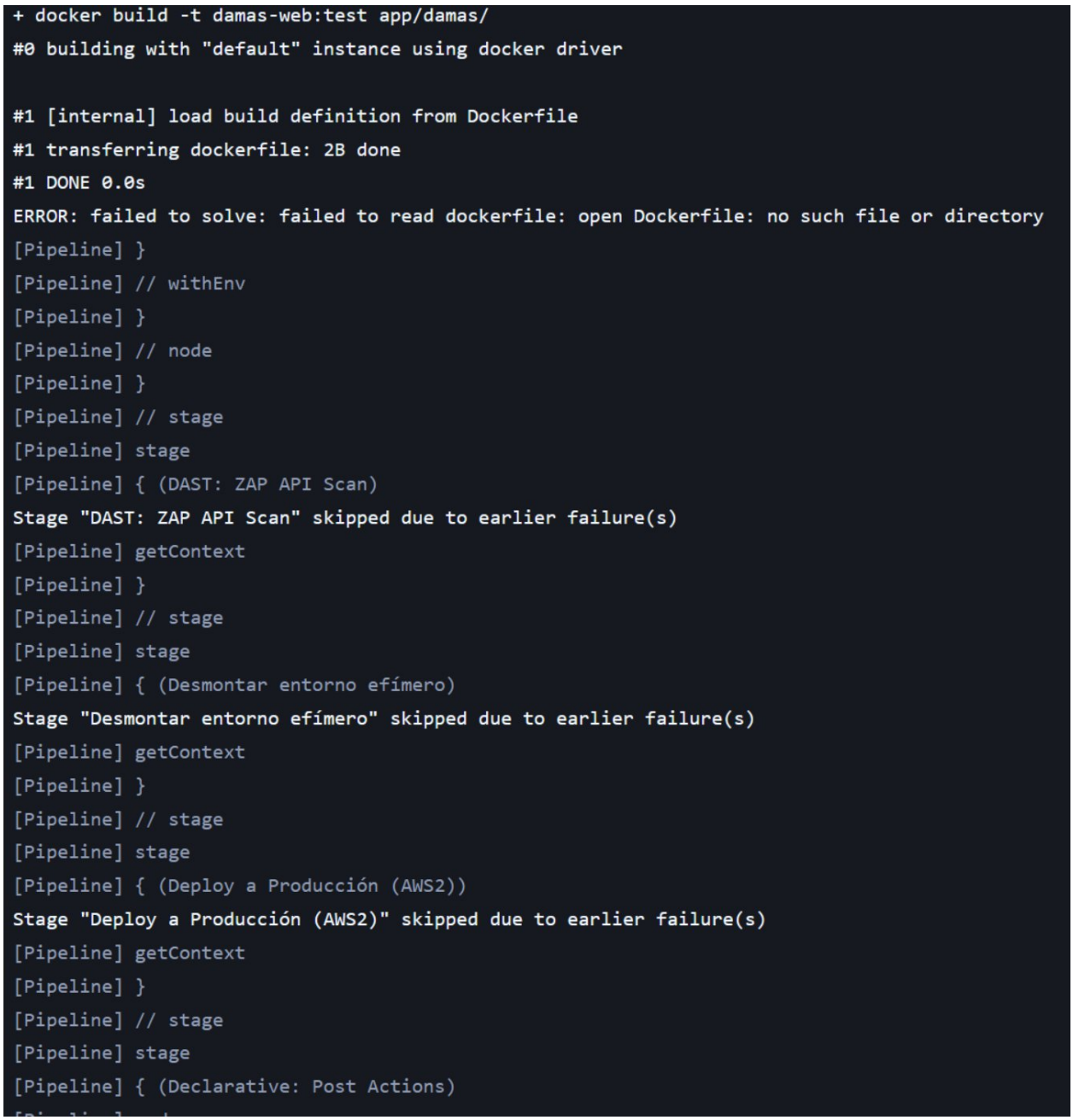
Primer fallo: no encuentra “damas-db”:

```
+ seq 1 30
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
mysqladmin: [Warning] Using a password on the command line interface can be insecure.
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
```

Jenkins intenta hacer ping a la base de datos de prueba (mysqladmin ping), pero Docker responde que no encuentra ese contenedor. ¿Por qué? Porque el comando docker run anterior, el que debía levantar el MySQL, falló en silencio.

El motivo está en esta línea de tu Jenkinsfile: **-v $PWD/db/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro** Jenkins intenta montar el archivo schema.sql desde una carpeta db/, pero según tu captura del repositorio en GitHub, tu schema.sql está directamente dentro de la carpeta app/damas/, no hay ninguna carpeta db. Como Docker no encuentra el archivo para montarlo, el contenedor de MySQL se aborta y se destruye al instante.

En este caso, solo es necesario cambiar la ruta en el Jenkinsfile.



Justo debajo de los errores de ping, vemos: ERROR: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory.

Jenkins intenta ejecutar docker build -t damas-web:test app/damas/. Le estamos diciendo a Docker que busque un archivo llamado exactamente Dockerfile dentro de la carpeta app/damas/. Sin embargo, revisando tu captura de GitHub otra vez, no hay ningún Dockerfile subido a ese repositorio, solo los archivos .php, .html, el schema.sql y el openapi.yaml.

Para esto, solo fue necesario subir el Dockerfile de AWS2 a app/damas/ de GitHub:

ProyectoASIR-Grupo-1 / app / damas / Dockerfile

sh1gehara Add Dockerfile for PHP 8.2 with Apache

Code Blame 18 lines (8 loc) · 395 Bytes

```
1   # Usamos la imagen oficial de PHP con Apache incorporado
2   FROM php:8.2-apache
3
4   # Instalamos la extensión mysqli para que PHP pueda hablar con MySQL
5   RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli
6
7   # Copiamos todos tus archivos PHP a la carpeta pública de Apache dentro del contenedor
8   COPY ./ /var/www/html/
9
10  # Le decimos a Apache que escuche por el puerto 80
11  EXPOSE 80
```

```
Stage "DAST: ZAP API Scan" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Desmontar entorno efímero)
Stage "Desmontar entorno efímero" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy a Producción (AWS2))
Stage "Deploy a Producción (AWS2)" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/TFG-Pipeline-Prueba
[Pipeline] {
[Pipeline] sh
+ docker rm -f damas-web damas-db
+ docker network rm damas-test
damas-test
[Pipeline] }
[Pipeline] // node
[Pipeline] echo
Pipeline Fallida — El código no ha alcanzado AWS2.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```

Fallo al finalizar la pipeline, aplicamos las soluciones a los problemas detectados y vamos con la segunda versión.

Segunda versión

```
+ seq 1 30
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
mysqladmin: [Warning] Using a password on the command line interface can be insecure.
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
Error response from daemon: No such container: damas-db
+ sleep 2
```

Al cambiar ambos, da error otra vez de no encontrar un “damas-db” de la stage de DAST.

Cambiamos el stage entero de DAST por lo siguiente, para detectar el error exacto:

```
stage('Levantar entorno efímero para DAST') {
        agent any
        steps {
            sh '''
                # Limpiamos restos de ejecuciones anteriores por si acaso
                docker network create damas-test 2>/dev/null || true
                docker rm -f damas-db damas-web 2>/dev/null || true

                # Ejecutamos SIN --rm para poder ver el error si muere
                docker run -d --name damas-db --network damas-test \
                  -e MYSQL_ROOT_PASSWORD=test \
                  -e MYSQL_DATABASE=damas \
                  -v $PWD/app/damas/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro \
                  mysql:8.0

                echo "Esperando a que la base de datos arranque..."
                for i in $(seq 1 30); do
                  # 1. Comprobamos si el contenedor ha muerto prematuramente
                  if ! docker ps -q -f name=damas-db | grep -q .; then
                    echo "EL CONTENEDOR HA CRASHEADO. Estos son los logs de MySQL:"
                    docker logs damas-db
                    exit 1
                  fi

                  # 2. Si sigue vivo, intentamos hacer ping
                  if docker exec damas-db mysqladmin ping -uroot -ptest --silent 2>/dev/null; then
                    echo "Base de datos lista y operativa."
                    break
                  fi
                  sleep 2
                done

                docker build -t damas-web:test app/damas/
                docker run -d --rm --name damas-web --network damas-test \
                  -e DB_HOST=damas-db \
                  -e DB_USER=root \
                  -e DB_PASS=test \
                  -e DB_NAME=damas \
                  damas-web:test
                sleep 3
            '''
        }
    }
```

Con un comando que nos suelte los logs del docker tenemos diagnóstico:

```
[Pipeline] {
[Pipeline] sh
+ docker network create damas-test
2d165bf12f8a77bcbe26490f549612b1910b0cc020b3cfcda7ff53e5cbd9bc1d
+ docker rm -f damas-db damas-web
+ docker run -d --name damas-db --network damas-test -e MYSQL_ROOT_PASSWORD=test -e MYSQL_DATABASE=damas -v /var/jenkins_home/workspace/TFG-Pipeline-Prueba/app/damas/
schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro mysql:8.0
43d73850cdf60cb05e7125d37c3d786d49bf736ec73e5818e967ca06950a6ccf
+ echo Esperando a que la base de datos arranque...
Esperando a que la base de datos arranque...
+ seq 1 30
+ docker ps -q -f name=damas-db
+ grep -q .
+ docker exec damas-db mysqladmin ping -uroot -ptest --silent
+ sleep 2
+ docker ps -q -f name=damas-db
+ grep -q .
+ echo EL CONTENEDOR HA CRASHEADO. Estos son los logs de MySQL:
EL CONTENEDOR HA CRASHEADO. Estos son los logs de MySQL:
+ docker logs damas-db
2026-05-17 00:19:05+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
2026-05-17 00:19:05+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-05-17 00:19:05+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
2026-05-17 00:19:05+00:00 [Note] [Entrypoint]: Initializing database files
2026-05-17T00:19:05.901881Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET
GLOBAL host_cache_size=0 instead.
2026-05-17T00:19:05.901935Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.46) initializing of server in progress as process 86
mysqld: Error writing file '/var/lib/mysql/auto.cnf' (OS errno 28 - No space left on device)
2026-05-17T00:19:05.904247Z 0 [ERROR] [MY-010076] [Server] Initialization of the server's UUID failed because it could not be read from the auto.cnf file. If this is
a new server, the initialization failed because it was not possible to generate a new UUID.
2026-05-17T00:19:05.904257Z 0 [ERROR] [MY-013236] [Server] The designated data directory /var/lib/mysql/ is unusable. You can remove all files that the server added
to it.
2026-05-17T00:19:05.904261Z 0 [ERROR] [MY-010119] [Server] Aborting
2026-05-17T00:19:05.904372Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.0.46)  MySQL Community Server - GPL.
+ exit 1
[Pipeline] }
```

Al parecer, el docker se ha quedado sin espacio de almacenamiento para poder hacer una BBDD.

Solución

En la máquina virtual de Ubuntu, donde está Jenkins (dentro de docker):

```
yamil@g1server-cicd:~/.ssh$ docker system prune -af --volumes
Deleted Containers:
b41b45ffec5349a8d0b702971779ff7b737cf1e46c45f29a6cf39dc2d845c751

Deleted Volumes:
4ad111f6fc09e0711ef9a9bb58766615ac5639d2b12eb34948786694986b5aa5

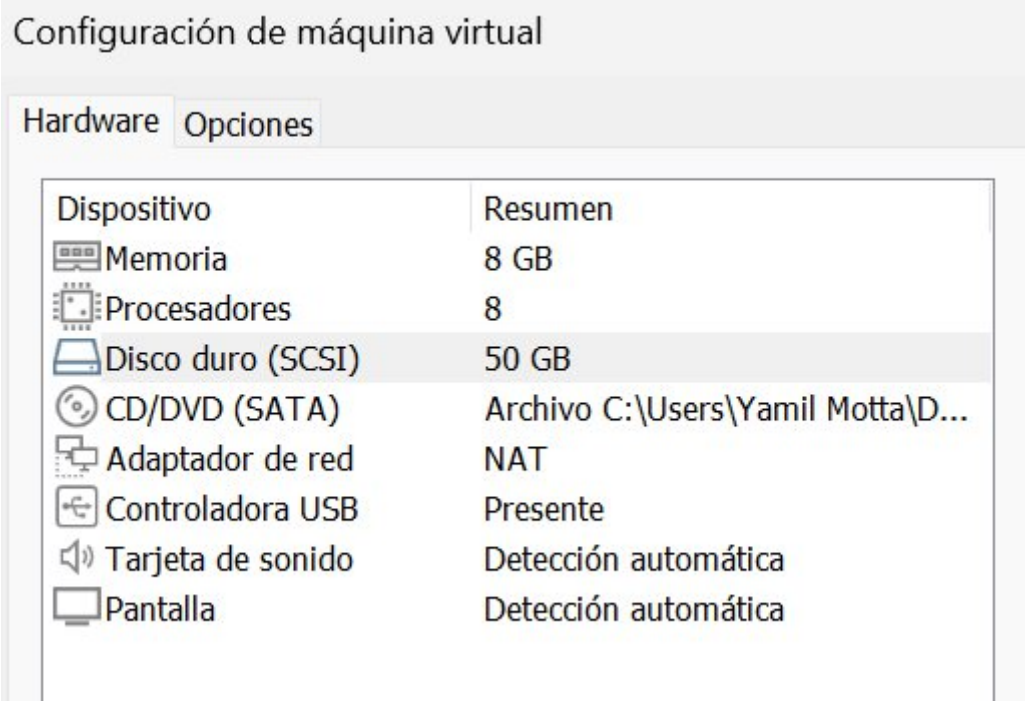
Deleted Images:
untagged: hello-world:latest
deleted: sha256:85404b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
deleted: sha256:2771e37a12b7bcb2902456ecf3f29bf9ee11ec348e66e8eb322d9780ad7fc2df
deleted: sha256:6b75187531c5e9b6a85c8946d5d82e4ef3801e051fbff338f382f3edfa60e3d2
untagged: mysql:8.0
deleted: sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2b
deleted: sha256:62fb722c78b24245ddff1796a0fcee4a49cc5b87e0aaaf20c92d1da9e0a2497b
deleted: sha256:fed8ae184a51c4b7d9cc23dd38a0a1bed605380eb6f11e0f45dd08c2db084d0a
untagged: php:8.2-cli
deleted: sha256:3694b70fc4f7d9f813b03248a9e796782cbdd5055dff5d192521846a37f41c8b
deleted: sha256:984959b53076f89c801da4e9d05b4b466b7bbeb84195d7aa941295284411d265
deleted: sha256:8f187c5151e24087aadffef87ae7f9fc49504adc10056c1dccc79a00428e21a5
untagged: returntocorp/semgrep:latest
deleted: sha256:9349edbadf90c3f3c0c3f55867625354e89680e6fa10d9034042af52fdb0e0d0
deleted: sha256:f682953ce85e3725f4a4dd94bd7ad13e570bb6b2c7a8cf7c6e38a9eac89239b2
deleted: sha256:402a8fe019dd46990cf082585baf772bb9fa74ced5067a62b288dbc6d18513de
untagged: ghcr.io/phpstan/phpstan:latest
deleted: sha256:8e72c05eebcfd439fcc3012a8d93f28ca72a706a3482d9149a0494b246db63f5
deleted: sha256:e0ece715695126b8a5bc43dc08002c990e67935bf8f527b379070055a6af4707
deleted: sha256:6fca63ef8557c0232cedad7fa20d21a159db353a8ff9d4652d117946b3366022

Deleted build cache objects:
s7jpspcuuduv6rzeq4o8tsi1b
qpl6zb5a9t1ftec59us847zs1
ummie7klfq9z1owlnnvcsb6fj
u81hrrdmxzgvxdut7b3100jz7
```

Borramos contenedores, redes, imágenes sin usar y sus cachés. Recuperamos casi 2GB.

```
Total reclaimed space: 1.964GB
yamil@g1server-cicd:~/.ssh$
```

Entonces, ampliamos la capacidad de almacenamiento de la máquina virtual de Jenkins:



Pasamos de 20GB a 50GB.

Encendemos la máquina de Ubuntu Server de nuevo y ampliamos el espacio:

```
yamil@g1server-cicd:~$ sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
[sudo] password for yamil:
  Size of logical volume ubuntu-vg/ubuntu-lv changed from 10,00 GiB (2560 extents) to 18,22 GiB (4665 ex
tents).
  Logical volume ubuntu-vg/ubuntu-lv successfully resized.
yamil@g1server-cicd:~$ sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/mapper/ubuntu--vg-ubuntu--lv is mounted on /; on-line resizing required
old_desc_blocks = 2, new_desc_blocks = 3
The filesystem on /dev/mapper/ubuntu--vg-ubuntu--lv is now 4776960 (4k) blocks long.

yamil@g1server-cicd:~$
```

```
yamil@g1server-cicd:~$ df -h
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                              790M  1,5M  788M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   18G  9,3G  7,8G  55% /
tmpfs                              3,9G     0  3,9G   0% /dev/shm
tmpfs                              5,0M     0  5,0M   0% /run/lock
/dev/sda2                          1,8G  200M  1,5G  13% /boot
tmpfs                              790M   12K  790M   1% /run/user/1000
yamil@g1server-cicd:~$
```

Ahora hay espacio para que los docker en la pipeline no se queden sin almacenamiento.

Tercera versión

Dockerfile

Una vez aumentado el almacenamiento, nos encontramos con otro problema: los drivers de PHP y cómo está configurado en el Dockerfile de AWS2:

ZAP no llega a encontrar los drivers (ni jenkins) de PHP:

```
[Pipeline] sh
+ mkdir -p zap-reports
+ docker run --rm --network damas-test curlimages/curl:latest -s -X POST -H Content-Type: application/json -d {"apodo_rojo":"R","apodo_negro":"N"} http://damas-web/
api.php?accion=nueva
Unable to find image 'curlimages/curl:latest' locally
latest: Pulling from curlimages/curl
b6066d233986: Pulling fs layer
b6066d233986: Download complete
b6066d233986: Pull complete
Digest: sha256:b3f1fb2a51d923260350d21b8654bbc607164a987e2f7c84a0ac199a67df812a
Status: Downloaded newer image for curlimages/curl:latest
+ PARTIDA={"ok":false,"error":"Error interno: could not find driver"}
+ echo {"ok":false,"error":"Error interno: could not find driver"}
+ grep -oP "codigo":"\K[^"]+
+ CODIGO=
```

Configuración Dockerfile nueva:

```
  GNU nano 7.2                                        Dockerfile *
# Usamos la imagen oficial de PHP con Apache incorporado
FROM php:8.2-apache

# Instalamos la extensión mysqli para que PHP pueda hablar con MySQL
RUN docker-php-ext-install mysqli pdo pdo_mysql && docker-php-ext-enable mysqli

# Copiamos todos tus archivos PHP a la carpeta pública de Apache dentro del contenedor
COPY ./ /var/www/html/

# Le decimos a Apache que escuche por el puerto 80
EXPOSE 80
```

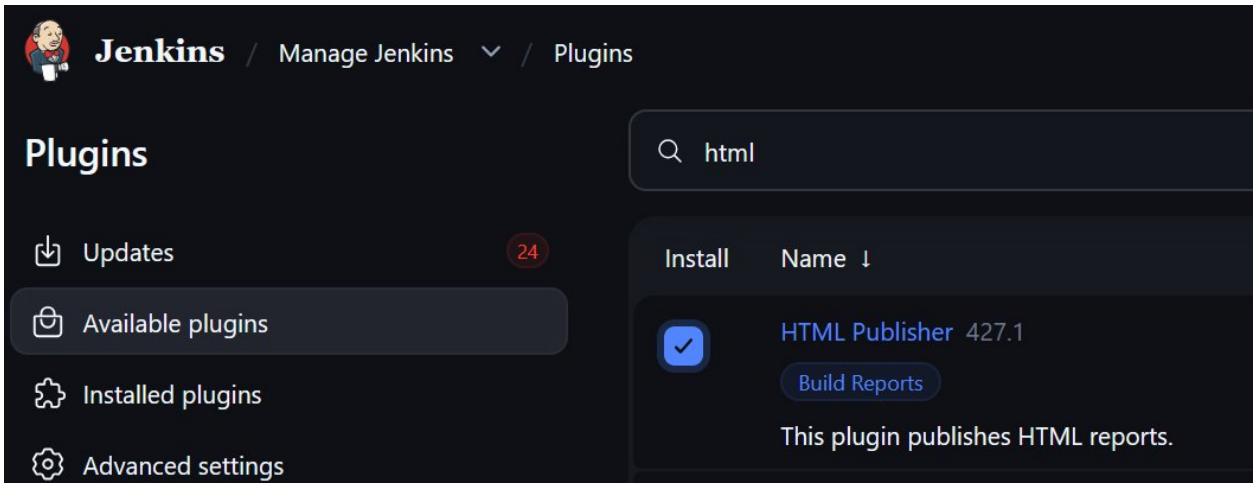
(Versión corregida con pdo y pdo_mysql en la misma línea).

HTML

Además, falta un plugin importante, el de hacer páginas HTML:

```
[Pipeline] archiveArtifacts
Archiving artifacts
Error when executing always post condition:
java.lang.NoSuchMethodError: No such DSL method 'publishHTML' found among steps [archive, bat, build, catchError, checkout, deleteDir, dir, dockerFingerprintFrom,
dockerFingerprintRun, echo, emailext, emailextrecipients, envVarsForTool, error, fileExists, findBuildScans, getContext, git, hideFromView, input, isUnix, junit,
library, libraryResource, load, mail, milestone, node, parallel, powershell, properties, publishChecks, pwd, pwsh, readFile, readScmFile, readTrusted, resolveScm,
retry, script, sh, sleep, stage, stash, step, timeout, timestamps, tm, tool, unarchive, unstable, unstash, validateDeclarativePipeline, waitForBuild, waitUntil,
warnError, withChecks, withContext, withCredentials, withDockerContainer, withDockerRegistry, withDockerServer, withEnv, withGradle, wrap, writeFile, ws] or symbols
[GitUsernamePassword, agent, all, allBranchesSame, allOf, always, ant, antFromApache, antOutcome, antTarget, any, anyOf, apiToken, apiTokenProperty, architecture,
archiveArtifacts, artifactManager, assembla, authorInChangelog, authorizationMatrix, aws, batchFile, bitbucket, bitbucketServer, booleanParam, branch,
brokenBuildSuspects, brokenTestsSuspects, browser, buildButton, buildDiscarder, buildDiscarders, buildRetention, buildSingleRevisionOnly, buildUser, buildingTag,
caseInsensitive, caseSensitive, certificate, cgit, changeRequest, changelog, changelogBase, changelogToBranch, changeset, checkoutOption, checkoutToSubdirectory,
choice, choiceParam, cleanAfterCheckout, cleanBeforeCheckout, cleanWs, clock, cloneOption, command, computerRetentionCheckInterval, consoleUrlProvider,
contentSecurityPolicy, contributor, cps, credentials, cron, crumb, culprits, dark, darkSystem, default, defaultDisplayUrlProvider, defaultFolderConfiguration,
defaultView, demand, developers, disableConcurrentBuilds, disableRestartFromStage, disableResume, discoverOtherRefs, discoverOtherRefsTrait, diskSpace,
diskSpaceMonitor, docker, dockerCert, dockerServer, dockerTool, dockerfile, downstream, dumb, durabilityHint, email-ext, envVars, envVarsFilter, environment, equals,
experimentalFlags, expression, extendedEmailPublisher, file, fileParam, filePath, fingerprint, fingerprints, firstBuildChangelog, fisheye, frameOptions, freeStyle,
freeStyleJob, fromDocker, fromScm, fromSource, git, gitBranchDiscovery, gitHooks, gitHubBranchDiscovery, gitHubBranchHeadAuthority, gitHubExcludeArchivedRepositories,
gitHubExcludeForkedRepositories, gitHubExcludePrivateRepositories, gitHubExcludePublicRepositories, gitHubForkDiscovery, gitHubIgnoreDraftPullRequestFilter,
```

En Manage Jenkins > Plugins:



Instalamos el plugin de HTML Publisher.

Cuarta versión



Esto nos confirma dos aspectos técnicos:

1. PHP está leyendo la variable damas-db a la perfección gracias a tu getenv().
2. La red interna de Docker es incapaz de encontrar a damas-db.

El motor de MySQL ejecuta un proceso de arranque dividido en dos fases. Primero, levanta un servicio temporal cerrado a la red para configurar contraseñas y procesar archivos de inicio, como tu schema.sql. Al terminar de leerlos, apaga ese motor temporal y levanta el servicio definitivo abriendo el puerto TCP.

Nuestro script de ping antiguo nos estaba engañando: detectaba el motor temporal, daba el visto bueno y Jenkins continuaba. Mientras Jenkins construía la imagen de la web, MySQL intentaba ejecutar tu schema.sql, se topaba con un error interno, y el contenedor moría en silencio. Para cuando la web intentaba lanzar el curl a la base de datos, el contenedor llevaba un rato apagado y su registro DNS había desaparecido.

Editamos el DAST de nuevo para que saque el error exacto del log:

```
stage('Levantar entorno efímero para DAST') {
        agent any
        steps {
            sh '''
                docker network create damas-test 2>/dev/null || true
                docker rm -f damas-db damas-web 2>/dev/null || true

                docker run -d --name damas-db --network damas-test \
                  -e MYSQL_ROOT_PASSWORD=test \
                  -e MYSQL_DATABASE=damas \
                  -v $PWD/app/damas/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro \
                  mysql:8.0

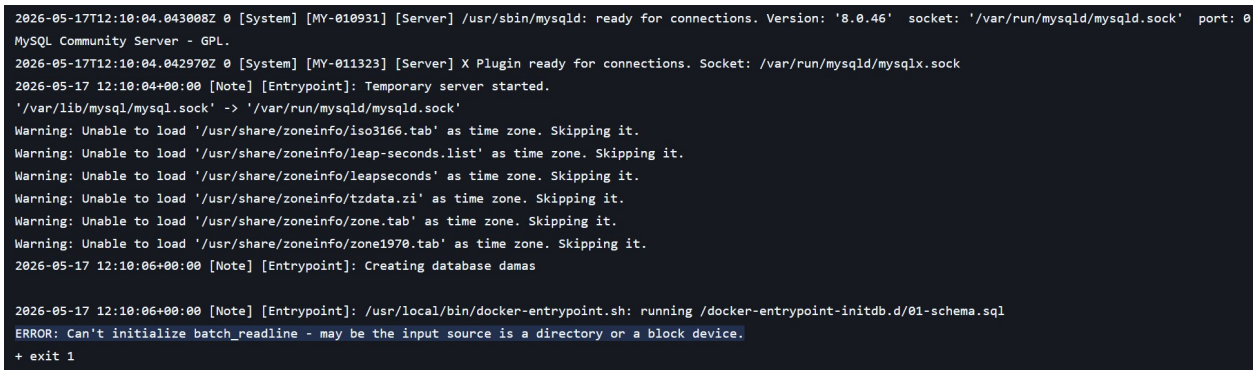
                echo "Esperando a que la base de datos arranque (Fase TCP)..."
                for i in $(seq 1 30); do
                  # Comprobar si el contenedor crashea ejecutando el schema.sql
                  if ! docker ps -q -f name=damas-db | grep -q .; then
                    echo "❌ EL CONTENEDOR MYSQL HA MUERTO. Mostrando logs:"
                    docker logs damas-db
                    exit 1
                  fi

                  # Forzar ping por TCP (ignora el servidor temporal)
                  if docker exec damas-db mysqladmin ping -uroot -ptest -h 127.0.0.1 --silent
2>/dev/null; then
                    echo "✅ Base de datos lista y operativa."
                    break
                  fi
                  sleep 2
                done

                docker build -t damas-web:test app/damas/
                docker run -d --rm --name damas-web --network damas-test \
                  -e DB_HOST=damas-db \
                  -e DB_USER=root \
                  -e DB_PASS=test \
                  -e DB_NAME=damas \
                  damas-web:test
                sleep 3
            '''
        }
    }
```

Quinta versión

En esta run se ve el siguiente error reseñable:



Básicamente, MySQL se está quejando de que el archivo 01-schema.sql que le hemos pasado no es un archivo de texto, sino una carpeta vacía.

Recordemos que Jenkins está montado sobre un Docker (DooD). Ahí radica el problema. Jenkins vive dentro de un contenedor, pero al ejecutar comandos de Docker, se los envía al motor (daemon) de tu máquina Ubuntu anfitriona.

Cuando el Jenkinsfile ejecuta la instrucción de montar el volumen (-v $PWD/app/damas/schema.sql:...), Jenkins traduce $PWD a su ruta interna: /var/jenkins_home/workspace/....

El motor de Docker en Ubuntu recibe esa orden, busca esa ruta en el disco duro de la máquina anfitriona y no la encuentra (porque esa ruta solo existe por dentro del contenedor de Jenkins). Como Docker está programado para crear los directorios que no existen cuando usamos -v, crea una carpeta vacía llamada schema.sql y se la enchufa a MySQL. MySQL intenta leerla como si fuera código SQL y muere instantáneamente.

Solución

La solución: Empaquetar en lugar de enlazar

En entornos DooD, la regla de oro es evitar montar volúmenes con código usando -v. La alternativa limpia y profesional es usar la herramienta docker build para meter el archivo directamente dentro de una imagen temporal. A docker build no le afectan los problemas de rutas porque empaqueta los archivos y se los manda por el socket al motor de Docker.

Vamos a cambiar el stage de la base de datos en el Jenkinsfile para que Jenkins cree un Dockerfile al vuelo, empaquete tu esquema SQL dentro de la imagen de MySQL y la levante sin usar volúmenes.

Volvemos a cambiar el stage de DAST:

```
stage('Levantar entorno efímero para DAST') {
        agent any
        steps {
            sh '''
                docker network create damas-test 2>/dev/null || true
                docker rm -f damas-db damas-web 2>/dev/null || true

                # 1. Empaquetamos la base de datos con el esquema dentro
                echo "FROM mysql:8.0" > Dockerfile.db
                echo "COPY app/damas/schema.sql /docker-entrypoint-initdb.d/01-schema.sql" >>
Dockerfile.db
                docker build -t damas-db:test -f Dockerfile.db .

                # 2. Levantamos la BD ya empaquetada (sin el parámetro -v)
                docker run -d --name damas-db --network damas-test \
                   -e MYSQL_ROOT_PASSWORD=test \
                   -e MYSQL_DATABASE=damas \
                   damas-db:test

                echo "Esperando a que la base de datos arranque (Fase TCP)..."
                for i in $(seq 1 30); do
                   if ! docker ps -q -f name=damas-db | grep -q .; then
                      echo "❌ EL CONTENEDOR MYSQL HA MUERTO. Mostrando logs:"
                      docker logs damas-db
                      exit 1
                   fi

                   if docker exec damas-db mysqladmin ping -uroot -ptest -h 127.0.0.1 --silent 2>/dev/null;
then
                      echo "✅ Base de datos lista y operativa."
                      break
                   fi
                   sleep 2
                done

                docker build -t damas-web:test app/damas/
                docker run -d --rm --name damas-web --network damas-test \
                   -e DB_HOST=damas-db \
                   -e DB_USER=root \
                   -e DB_PASS=test \
                   -e DB_NAME=damas \
                   damas-web:test
                sleep 3
            '''
        }
    }
```

Sexta versión

Esta vez, el problema lo da ZAP:

```
[Pipeline] {
[Pipeline] sh
+ mkdir -p zap-reports
+ docker run --rm --network damas-test curlimages/curl:latest -s -X POST -H Content-Type: application/json -d {"apodo_rojo":"R","apodo_negro":"N"} http://damas-web/
api.php?accion=nueva
+ PARTIDA={"ok":true,"id":1}
+ echo {"ok":true,"id":1}
+ grep -oP "codigo":"\K[^"]+
+ CODIGO=
Post stage
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] publishHTML
[htmlpublisher] Archiving HTML reports...
[htmlpublisher] Archiving at BUILD level /var/jenkins_home/workspace/TFG-Pipeline-Prueba/zap-reports to OWASP_20ZAP_20Report
[htmlpublisher] Copying recursive using current thread
ERROR: Directory '/var/jenkins_home/workspace/TFG-Pipeline-Prueba/zap-reports' exists but failed copying to 'OWASP_20ZAP_20Report'.
[Pipeline] }
[Pipeline] // withEnv
```

Cuando el script de Jenkins hace la petición web para iniciar la partida, espera recibir un JSON que contenga exactamente los campos "codigo" y "token_rojo" para extraerlos y pasárselos a ZAP.

Como la API en este momento ha respondido únicamente con el identificador ("id":1), el comando grep encargado de buscar la palabra "codigo" no encuentra coincidencias. Al fallar la búsqueda, la consola de Linux devuelve un error y Jenkins aborta la ejecución inmediatamente por precaución.

Solución

Para que las herramientas de automatización no se vuelvan frágiles ante pequeños cambios en el formato de la API, la práctica habitual es añadir un "salvavidas". Si el script no encuentra el dato exacto que busca, le inyectamos un valor por defecto (un placeholder). De esta forma, OWASP ZAP recibe datos válidos para seguir haciendo sus pruebas de seguridad y la pipeline no se detiene.

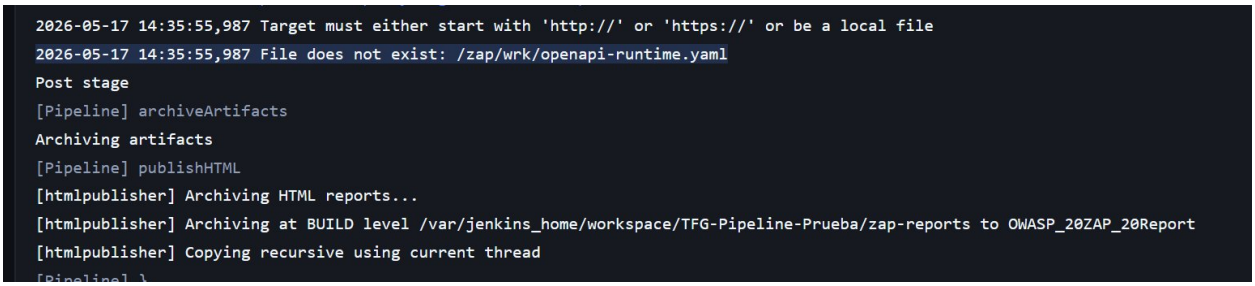
Actualizamos el stage DAST: ZAP API Scan

```
# Interceptar variables de una partida real para el fuzzing
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                   -s -X POST -H "Content-Type: application/json" \
                   -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                   "http://damas-web/api.php?accion=nueva")

                # Añadimos un valor por defecto (|| echo) para evitar crasheos si el JSON cambia
                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+' || echo "abc12345")
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+' || echo
"00000000000000000000000000000000000000000000000000000000000000000000")
```

Séptima versión

El DB.php, GameLogic.php y api.php se suben actualizados a github, versión con tokens que ya solucionó Saskia en AWS2. Se actualiza también el schema.sql a la versión actual (ubicado en MySQL-Server tras la subred privada). Esta vez, el error lo da al eliminar el comando “sed” y no tener un runtime del openapi:



La causa de esto ha sido un problema de comunicación. Al pasar el bloque de código corregido para la extracción de variables (PARTIDA, CODIGO y TOKEN_R), al sustituirlo en el Jenkinsfile, se ha borrado la herramienta sed que venía justo debajo. Ese comando sed era el encargado de coger tu openapi.yaml original, inyectarle el código y el token reales que acabamos de extraer, y guardarlo en la carpeta de reportes bajo el nombre openapi-runtime.yaml. Como no se ha ejecutado, el archivo no existe y ZAP choca contra un muro.

```
stage('DAST: ZAP API Scan') {
        agent any
        steps {
            sh '''
                mkdir -p zap-reports

                # Interceptar variables de una partida real para el fuzzing
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                    -s -X POST -H "Content-Type: application/json" \
                    -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                    "http://damas-web/api.php?accion=nueva")

                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                # Inyectar variables en el OpenAPI
                sed -e "s/abc12345/$CODIGO/g" \
                    -e
"s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g" \
                    app/damas/openapi.yaml > zap-reports/openapi-runtime.yaml

                # Ejecutar escaneo activo
                docker run --rm --network damas-test \
                    -v $PWD/zap-reports:/zap/wrk:rw \
                    ghcr.io/zaproxy/zaproxy:stable \
                    zap-api-scan.py \
                        -t /zap/wrk/openapi-runtime.yaml \
                        -f openapi \
                        -r zap-api-report.html \
                        -J zap-api-report.json \
                        -I
            '''
        }
        post {
            always {
                archiveArtifacts artifacts: 'zap-reports/*', allowEmptyArchive: true
```

```
                publishHTML target: [
                    reportDir: 'zap-reports',
                    reportFiles: 'zap-api-report.html',
                    reportName: 'OWASP ZAP Report',
                    keepAll: true
                ]
            }
        }
    }
```

Con esto hecho, vamos a la octava versión.

Octava versión

El sed en diferentes lineas puede ser que rompa el jenkins, simplificamos el sed en una sola línea:

```
stage('DAST: ZAP API Scan') {
            agent any
            steps {
                sh '''
                    mkdir -p zap-reports

                    # Interceptar variables de una partida real para el fuzzing
                    PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                        -s -X POST -H "Content-Type: application/json" \
                        -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                        "http://damas-web/api.php?accion=nueva")

                    CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                    TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                    # Inyectar variables en el OpenAPI
                    sed -e "s/abc12345/$CODIGO/g" -e "s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g" app/damas/openapi.yaml > zap-reports/openapi-runtime.yaml

                    # Ejecutar escaneo activo
                    docker run --rm --network damas-test \
```

Novena versión

```
96b61ea0bd73fd5c611c7201e2a851d","apodo_rojo":"R","apodo_negro":"N"}
+ grep -oP "token_rojo":"\K[^"]+
+ TOKEN_R=081825d8a2df2a8656687139571bb8d3859b96b5f39db698070e92e5a25d299d
+ sed -e s/abc12345/35D68274/g -e s/0000000000000000000000000000000000000000000000000000000000000000/081825d8a2df2a8656687139571bb8d3859b96b5f39db698070e92e5a25d299d/
g app/damas/openapi.yaml
+ docker run --rm --network damas-test -v /var/jenkins_home/workspace/TFG-Pipeline-Prueba/zap-reports:/zap/wrk:rw ghcr.io/zaproxy/zaproxy:stable zap-api-scan.py -t /
zap/wrk/openapi-runtime.yaml -f openapi -r zap-api-report.html -J zap-api-report.json -I
2026-05-17 15:02:14,343 Target must either start with 'http://' or 'https://' or be a local file
2026-05-17 15:02:14,344 File does not exist: /zap/wrk/openapi-runtime.yaml
Usage: zap-api-scan.py -t <target> -f <format> [options]
    -t target         target API definition, OpenAPI or SOAP, local file or URL, e.g. https://www.example.com/openapi.json
```

Da error de no encontrar el openapi-runtine.yaml, el problema es el siguiente:

```
# Ejecutar escaneo activo
docker run --rm --network damas-test \
    -v $PWD/zap-reports:/zap/wrk:rw \
    ghcr.io/zaproxy/zaproxy:stable \
    zap-api-scan.py \
        -t /zap/wrk/openapi-runtime.yaml \
        -f openapi \
        -r zap-api-report.html \
        -J zap-api-report.json \
        -I
```

Cuando Jenkins le envía ese comando al motor Docker de la máquina Ubuntu anfitriona, le está diciendo: "Coge esta ruta de Jenkins y métela en el escáner ZAP". Como el motor Docker de Ubuntu no encuentra esa ruta en su propio disco duro, crea una carpeta vacía y se la enchufa a ZAP. ZAP mira dentro de la carpeta, ve que no hay ningún archivo, y lanza el error File does not exist: /zap/wrk/openapi-runtime.yaml.

Solución

La mejor práctica en arquitecturas DooD para este caso es usar el comando docker cp. Este comando permite copiar archivos entre tu contenedor de Jenkins y el contenedor de ZAP a través del "socket", esquivando por completo el problema de las rutas del disco duro.

```
    # 1. Crear el contenedor de ZAP (sin encenderlo) y sin montar volúmenes (-v)
    docker rm -f zap-scanner 2>/dev/null || true
    docker create --name zap-scanner --network damas-test \
        ghcr.io/zaproxy/zaproxy:stable \
        zap-api-scan.py \
            -t /zap/wrk/openapi-runtime.yaml \
            -f openapi \
            -r zap-api-report.html \
            -J zap-api-report.json \
            -I

    # 2. Copiar el archivo configurado desde Jenkins hacia dentro de ZAP
    docker cp openapi-runtime.yaml zap-scanner:/zap/wrk/

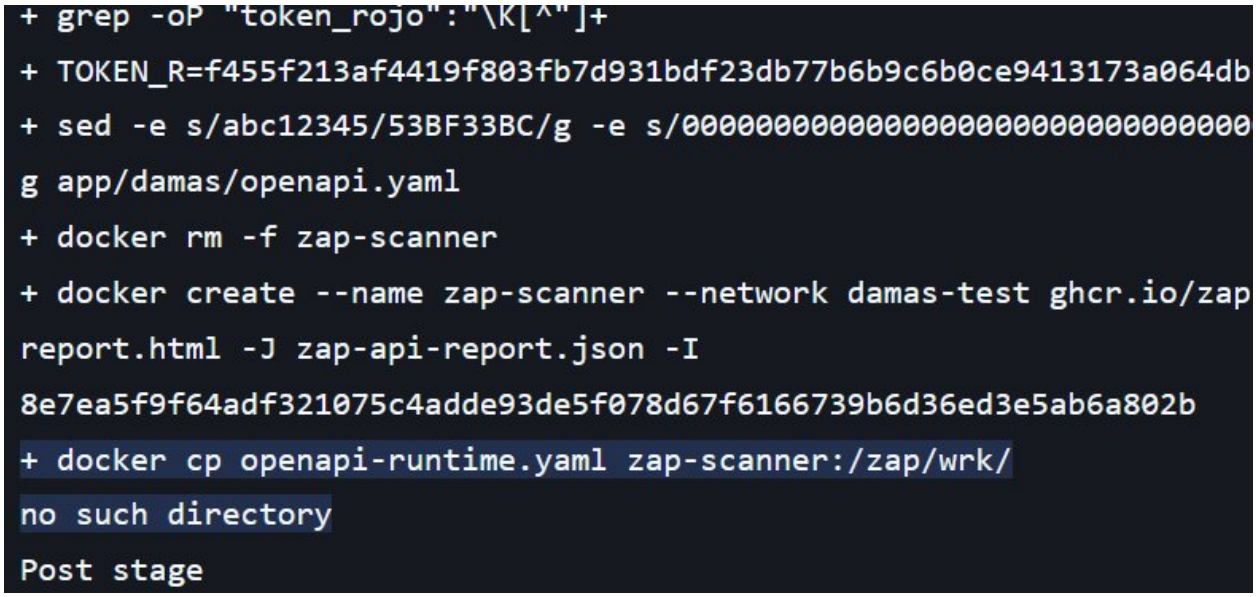
    # 3. Arrancar ZAP y esperar a que termine su ataque
    docker start -a zap-scanner

    # 4. Extraer los reportes generados de vuelta a Jenkins
    docker cp zap-scanner:/zap/wrk/zap-api-report.html zap-reports/ || true
    docker cp zap-scanner:/zap/wrk/zap-api-report.json zap-reports/ || true

    # 5. Limpieza
    docker rm -f zap-scanner
'''
```

Décima versión

El error en esta versión en el stage ZAP API Scan es:



Esto ocurre por un detalle de cómo está construida la imagen oficial de ZAP. La carpeta /zap/wrk/ no existe por defecto en el sistema de archivos del contenedor. Es un punto de montaje que el motor de Docker crea al vuelo automáticamente solo cuando le pasas el parámetro -v. Como en la ejecución anterior eliminamos ese parámetro para sortear el problema de las rutas de tu máquina anfitriona, al intentar usar cp, Linux rechaza la copia porque la carpeta de destino no está ahí.

Solución

Para solucionarlo, vamos a depositar el archivo directamente en la raíz de trabajo principal de la herramienta, que es la carpeta /zap/, la cual siempre existe.

```
stage('DAST: ZAP API Scan') {
        agent any
        steps {
            sh '''
                mkdir -p zap-reports

                # Interceptar variables de una partida real
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                    -s -X POST -H "Content-Type: application/json" \
                    -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                    "http://damas-web/api.php?accion=nueva")

                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                # Inyectar variables en el OpenAPI
                sed -e "s/abc12345/$CODIGO/g" -e
"s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g"
app/damas/openapi.yaml > openapi-runtime.yaml

                # 1. Crear el contenedor de ZAP apuntando a la ruta /zap/
                docker rm -f zap-scanner 2>/dev/null || true
                docker create --name zap-scanner --network damas-test \
                    ghcr.io/zaproxy/zaproxy:stable \
                    zap-api-scan.py \
```

```
                    -t /zap/openapi-runtime.yaml \
                    -f openapi \
                    -r zap-api-report.html \
                    -J zap-api-report.json \
                    -l

                # 2. Copiar el archivo directamente a la carpeta /zap/
                docker cp openapi-runtime.yaml zap-scanner:/zap/openapi-runtime.yaml

                # 3. Arrancar ZAP y esperar a que termine su ataque
                docker start -a zap-scanner

                # 4. Extraer los reportes desde /zap/
                docker cp zap-scanner:/zap/zap-api-report.html zap-reports/ || true
                docker cp zap-scanner:/zap/zap-api-report.json zap-reports/ || true

                # 5. Limpieza
                docker rm -f zap-scanner
            '''
        }
        post {
            always {
                archiveArtifacts artifacts: 'zap-reports/*', allowEmptyArchive: true
                publishHTML target: [
                    reportDir: 'zap-reports',
                    reportFiles: 'zap-api-report.html',
                    reportName: 'OWASP ZAP Report',
                    keepAll: true
                ]
            }
        }
    }
```

Undécima versión

```
For more details see https://www.zaproxy.org/docs/docker/api-scan/
2026-05-17 15:49:02,816 A file based option has been specified but the directory '/zap/wrk' is not mounted
Post stage
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] publishHTML
[htmlpublisher] Archiving HTML reports...
[htmlpublisher] Archiving at BUILD level /var/jenkins_home/workspace/TFG-Pipeline-Prueba/zap-reports to OWASP_20ZAP_20Report
[htmlpublisher] Copying recursive using current thread
[Pipeline] }
```

Esta vez, nos dice que /zap/wrk tiene que ser un volumen montado, y no una carpeta como habíamos configurado antes.

Resulta que los desarrolladores de OWASP programaron el script zap-api-scan.py con una comprobación estricta de seguridad: si le pides que genere reportes usando parámetros como -r o -J, el script verifica físicamente si la ruta /zap/wrk es un volumen montado. Si detecta que es una simple carpeta (como hicimos nosotros copiando el archivo directamente), aborta la ejecución al instante y lanza ese error.

Solución

Para solucionar esto en una arquitectura DooD (donde no podemos montar rutas locales del disco con -v $PWD), la técnica estándar es utilizar Volúmenes Nombrados (Named Volumes). Es un espacio de almacenamiento que Docker gestiona internamente, por lo que no le afectan los problemas de rutas.

Vamos a crear un volumen, usaremos un contenedor temporal (un helper muy ligero) para inyectar tu archivo ahí dentro, y luego le pasaremos ese volumen a ZAP. Así ZAP detectará el montaje correctamente y no se quejará.

```
stage('DAST: ZAP API Scan') {
        agent any
        steps {
            sh '''
                mkdir -p zap-reports

                # Interceptar variables de una partida real
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                    -s -X POST -H "Content-Type: application/json" \
                    -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                    "http://damas-web/api.php?accion=nueva")

                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                # Inyectar variables en el OpenAPI
                sed -e "s/abc12345/$CODIGO/g" -e
"s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g" app/damas/openapi.yaml >
openapi-runtime.yaml

                # 1. Crear un volumen Docker puro (gestionado por el motor, sin problemas de rutas)
                docker volume create zap-data

                # 2. Usar un contenedor auxiliar (helper) para inyectar el archivo en el volumen
                docker create --name zap-helper -v zap-data:/zap/wrk alpine tail
                docker cp openapi-runtime.yaml zap-helper:/zap/wrk/

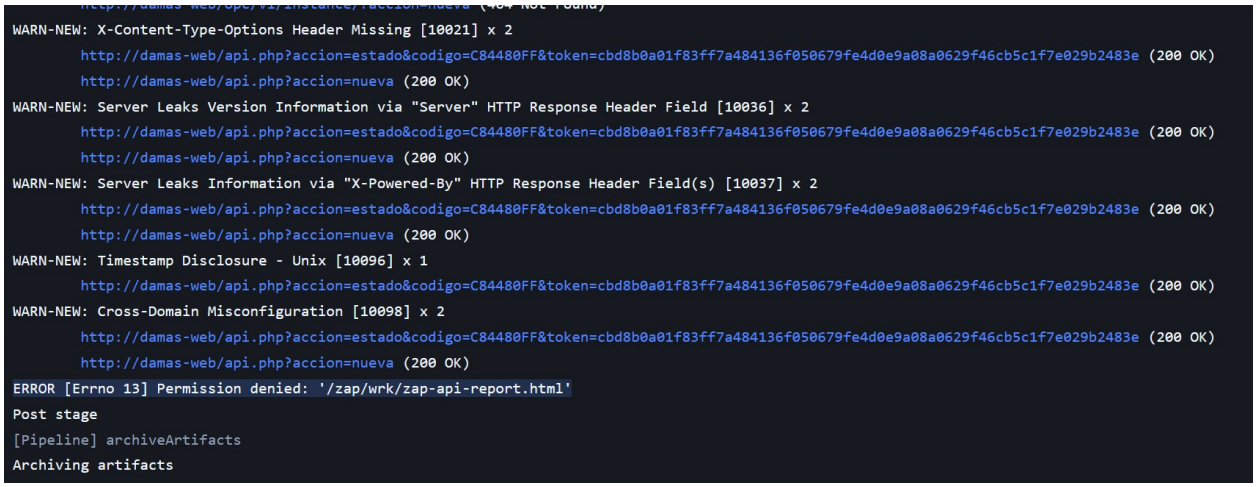
                # 3. Lanzar ZAP montando el volumen. (Ahora sí pasará su comprobación estricta)
                docker run --rm --network damas-test -v zap-data:/zap/wrk \
                    ghcr.io/zaproxy/zaproxy:stable \
                    zap-api-scan.py \
                        -t /zap/wrk/openapi-runtime.yaml \
                        -f openapi \
                        -r zap-api-report.html \
                        -J zap-api-report.json \
                        -I

                # 4. Extraer los reportes generados desde el volumen a nuestro Jenkins
                docker cp zap-helper:/zap/wrk/zap-api-report.html zap-reports/ || true
                docker cp zap-helper:/zap/wrk/zap-api-report.json zap-reports/ || true

                # 5. Limpiar el entorno
                docker rm -f zap-helper
                docker volume rm zap-data
            '''
        }
        post {
            always {
                archiveArtifacts artifacts: 'zap-reports/*', allowEmptyArchive: true
                publishHTML target: [
                    reportDir: 'zap-reports',
                    reportFiles: 'zap-api-report.html',
                    reportName: 'OWASP ZAP Report',
                    keepAll: true
                ]
            }
        }
    }
```

Duodécima versión

Esta vez, pasamos de los escaneos de la API:



Este obstáculo está relacionado con el aislamiento de seguridad (sandbox) de la propia herramienta de OWASP. Para evitar que el escáner sea comprometido, el contenedor de ZAP no se ejecuta como administrador (root), sino bajo un usuario con privilegios mínimos.

Sin embargo, el volumen de almacenamiento zap-data que creamos en el paso anterior pertenece a root. El usuario restringido de ZAP puede leer tus instrucciones OpenAPI que están dentro, pero Linux le deniega el permiso cuando intenta guardar (escribir) los reportes de vulnerabilidades en ese mismo espacio.

Solución

Para solucionarlo, solo necesitamos usar un contenedor rápido de Alpine justo antes de arrancar ZAP para abrir los permisos de escritura del volumen (chmod 777).

```
stage('DAST: ZAP API Scan') {
        agent any
        steps {
            sh '''
                mkdir -p zap-reports

                # Interceptar variables de una partida real
                PARTIDA=$(docker run --rm --network damas-test curlimages/curl:latest \
                   -s -X POST -H "Content-Type: application/json" \
                   -d '{"apodo_rojo":"R","apodo_negro":"N"}' \
                   "http://damas-web/api.php?accion=nueva")

                CODIGO=$(echo $PARTIDA | grep -oP '"codigo":"\\K[^"]+')
                TOKEN_R=$(echo $PARTIDA | grep -oP '"token_rojo":"\\K[^"]+')

                # Inyectar variables en el OpenAPI
                sed -e "s/abc12345/$CODIGO/g" -e
"s/0000000000000000000000000000000000000000000000000000000000000000/$TOKEN_R/g" app/damas/openapi.yaml >
openapi-runtime.yaml

                # 1. Crear el volumen limpio
                docker volume rm zap-data 2>/dev/null || true
                docker volume create zap-data

                # 2. Usar un contenedor auxiliar (helper) para inyectar el archivo en el volumen
                docker rm -f zap-helper 2>/dev/null || true
                docker create --name zap-helper -v zap-data:/zap/wrk alpine tail
                docker cp openapi-runtime.yaml zap-helper:/zap/wrk/

                # 3. ASIGNAR PERMISOS: Dar acceso de escritura al usuario restringido de ZAP
                docker run --rm -v zap-data:/zap/wrk alpine chmod 777 /zap/wrk

                # 4. Lanzar ZAP montando el volumen
```

```
            docker run --rm --network damas-test -v zap-data:/zap/wrk \
              ghcr.io/zaproxy/zaproxy:stable \
              zap-api-scan.py \
                -t /zap/wrk/openapi-runtime.yaml \
                -f openapi \
                -r zap-api-report.html \
                -J zap-api-report.json \
                -l

            # 5. Extraer los reportes generados desde el volumen a nuestro Jenkins
            docker cp zap-helper:/zap/wrk/zap-api-report.html zap-reports/ || true
            docker cp zap-helper:/zap/wrk/zap-api-report.json zap-reports/ || true

            # 6. Limpiar el entorno
            docker rm -f zap-helper
            docker volume rm zap-data
          '''
        }
      }
      post {
        always {
          archiveArtifacts artifacts: 'zap-reports/*', allowEmptyArchive: true
          publishHTML target: [
            reportDir: 'zap-reports',
            reportFiles: 'zap-api-report.html',
            reportName: 'OWASP ZAP Report',
            keepAll: true
          ]
        }
      }
    }
```

Trigésima versión

Finalmente, la pipeline da SUCCESS:

```
Stage "Deploy a Producción (AWS2)" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/TFG-Pipeline-Prueba
[Pipeline] {
[Pipeline] sh
+ docker rm -f damas-web damas-db
+ docker network rm damas-test
+ true
[Pipeline] }
[Pipeline] // node
[Pipeline] echo
Pipeline OK — Despliegue completado.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] End of Pipeline
Finished: SUCCESS
```

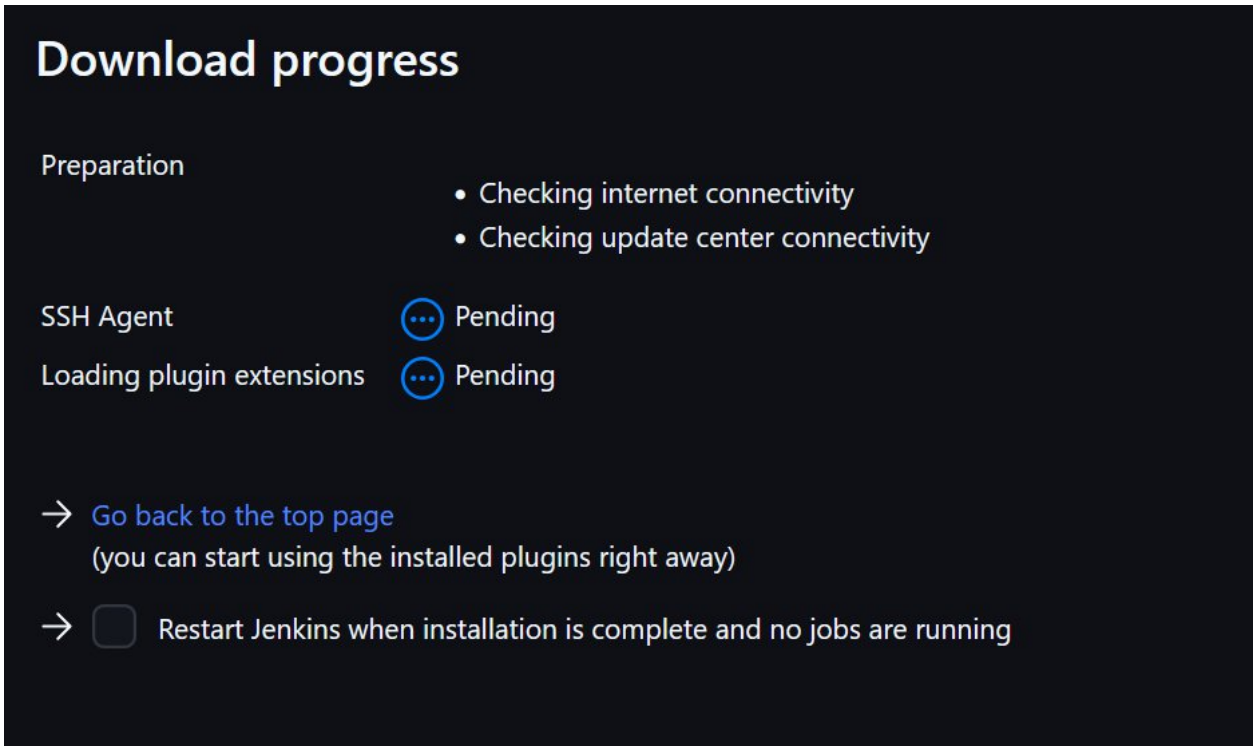
Sin embargo, se salta el despliegue por la condicional “when branch: main”.

Ejecutamos la pipeline sin la condicional:

```
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source)
        at java.base/java.lang.reflect.Method.invoke(Unknown Source)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.ContinuationPtr$ContinuationImpl.receive(ContinuationPtr.java:71)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.ClosureBlock.eval(ClosureBlock.java:49)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.Next.step(Next.java:84)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.Continuable.run0(Continuable.java:142)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.SandboxContinuable.access$001(SandboxContinuable.java:17)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.SandboxContinuable.run0(SandboxContinuable.java:48)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThread.runNextChunk(CpsThread.java:188)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup.run(CpsThreadGroup.java:464)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup$2.call(CpsThreadGroup.java:372)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup$2.call(CpsThreadGroup.java:302)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService.lambda$wrap$4(CpsVmExecutorService.java:143)
        at java.base/java.util.concurrent.FutureTask.run(Unknown Source)
        at hudson.remoting.SingleLaneExecutorService$1.run(SingleLaneExecutorService.java:139)
        at jenkins.util.ContextResettingExecutorService$1.run(ContextResettingExecutorService.java:28)
        at jenkins.security.ImpersonatingExecutorService$1.run(ImpersonatingExecutorService.java:68)
        at jenkins.util.ErrorLoggingExecutorService.lambda$wrap$0(ErrorLoggingExecutorService.java:51)
        at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Unknown Source)
        at java.base/java.util.concurrent.FutureTask.run(Unknown Source)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(Unknown Source)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(Unknown Source)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService$1.call(CpsVmExecutorService.java:53)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService$1.call(CpsVmExecutorService.java:50)
        at org.codehaus.groovy.runtime.GroovyCategorySupport$ThreadCategoryInfo.use(GroovyCategorySupport.java:136)
        at org.codehaus.groovy.runtime.GroovyCategorySupport.use(GroovyCategorySupport.java:275)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService.lambda$categoryThreadFactory$0(CpsVmExecutorService.java:50)
        at java.base/java.lang.Thread.run(Unknown Source)
Finished: FAILURE
```

Falta de plugins, en concreto sshagent.

En Manage Jenkins > Plugins:



Décimocuarta versión

Esta vez falla el rsync, no está instalado en Docker:

```
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-4ixLpgj6BtYk/agent.5611
SSH_AGENT_PID=5614
Running ssh-add (command line suppressed)
Identity added: /var/jenkins_home/workspace/TFG-Pipeline-Prueba@tmp/private_key_3702825449594664367.key (jenkins@tfg-cicd)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ rsync -avz --delete -e ssh -o StrictHostKeyChecking=accept-new app/damas/html/ ubuntu@10.100.0.3:/home/ubuntu/mi_proyecto_web/html/
/var/jenkins_home/workspace/TFG-Pipeline-Prueba@tmp/durable-4061f71a/script.sh.copy: 2: rsync: not found
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
```

En la propia máquina ubuntu server de jenkins (en el contenedor docker), se instala el rsync:

```
yamil@g1server-cicd:~/tfg/jenkins$ docker exec -u 0 jenkins_core apt-get update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [163 kB]
Fetched 10.1 MB in 10s (971 kB/s)
Reading package lists...
yamil@g1server-cicd:~/tfg/jenkins$ docker exec -u 0 jenkins_core apt-get install -y rsync
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  libpopt0
Suggested packages:
  openssh-server python3-braceexpand
The following NEW packages will be installed:
  libpopt0 rsync
0 upgraded, 2 newly installed, 0 to remove and 42 not upgraded.
Need to get 473 kB of archives.
After this operation, 1080 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main amd64 libpopt0 amd64 1.19+dfsg-2 [43.8 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 rsync amd64 3.4.1+ds1-5+deb13u2 [429 kB]
```

Además, ajustar la ruta de app/damas (anteriormente app/damas/html, que es inexistente en el repositorio)

```
stage('Deploy a Producción (AWS2)') {
    agent any
    steps {
        sshagent(['aws2-deploy-ssh-key']) {
            sh '''
                rsync -avz --delete \
                    -e "ssh -o StrictHostKeyChecking=accept-new" \
                    app/damas/ \
                    ubuntu@10.100.0.3:/home/ubuntu/mi_proyecto_web/html/

                ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 \
                    'cd /home/ubuntu/mi_proyecto_web && docker compose up -d --build'
            '''
        }
```

Decimoquinta versión

Ahora Jenkins interpreta el comando

ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 cd /home/ubuntu/mi_proyecto_web && docker compose up -d --build

Como un comando de Jenkins en lugar de uno de la máquina virtual.

```
+ rsync -avz --delete -e ssh -o StrictHostKeyChecking=accept-new app/damas/ ubuntu@10.100.0.3:/home/ubuntu/mi_proyecto_web/html/
Warning: Permanently added '10.100.0.3' (ED25519) to the list of known hosts.
sending incremental file list
./
DB.php
Dockerfile
GameLogic.php
api.php
index.html
nada.txt
openapi.yaml
schema.sql

sent 6,149 bytes  received 525 bytes  1,906.86 bytes/sec
total size is 58,380  speedup is 8.75
+ ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 cd /home/ubuntu/mi_proyecto_web && docker compose up -d --build
unknown shorthand flag: 'd' in -d

Usage:  docker [OPTIONS] COMMAND [ARG...]
```

Esta vez se modifica con unas comillas para que Jenkins sepa que es una instrucción para la máquina virtual:

```
ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 "cd /home/ubuntu/mi_proyecto_web && docker compose up -d --build"
```

Decimosexta versión

Jenkins se queda sin espacio en disco, debido a no reubicar el nuevo espacio a la partición actual, se añaden bien los 50GB de espacio en el disco desde la terminal de Ubuntu Server:

```
yamil@g1server-cicd:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   50G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0  1,8G  0 part /boot
└─sda3                      8:3    0 18,2G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0 18,2G  0 lvm  /
sr0                        11:0    1  3,2G  0 rom
yamil@g1server-cicd:~$ sudo pvs
  PV         VG        Fmt  Attr PSize  PFree
  /dev/sda3  ubuntu-vg lvm2 a--  18,22g    0
yamil@g1server-cicd:~$ sudo vgs
  VG        #PV #LV #SN Attr   VSize  VFree
  ubuntu-vg   1   1   0 wz--n- 18,22g    0
yamil@g1server-cicd:~$ sudo lvs
  LV        VG        Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  ubuntu-lv ubuntu-vg -wi-ao---- 18,22g
yamil@g1server-cicd:~$ sudo growpart /dev/sda 3
CHANGED: partition=3 start=3719168 old: size=38221824 end=41940991 new: size=101138399 end=104857566
yamil@g1server-cicd:~$ sudo pvresize /dev/sda3
  Physical volume "/dev/sda3" changed
  1 physical volume(s) resized or updated / 0 physical volume(s) not resized
```

Se ve el estado actual de disco y se extiende la partición física.

```
yamil@g1server-cicd:~$ sudo lvextend -l +100%FREE /dev/mapper/ubuntu--vg-ubuntu--lv
  Size of logical volume ubuntu-vg/ubuntu-lv changed from 18,22 GiB (4665 extents) to 48,22 GiB (12345 e
xtents).
  Logical volume ubuntu-vg/ubuntu-lv successfully resized.
yamil@g1server-cicd:~$ sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/mapper/ubuntu--vg-ubuntu--lv is mounted on /; on-line resizing required
old_desc_blocks = 3, new_desc_blocks = 7
The filesystem on /dev/mapper/ubuntu--vg-ubuntu--lv is now 12641280 (4k) blocks long.

yamil@g1server-cicd:~$ df -h /
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   48G  5,5G   40G  13% /
yamil@g1server-cicd:~$
```

Se extiende la partición lógica al 100% del espacio libre; se extiende el filesystem (ext4) en caliente, sin desmontar.

Ahora, Jenkins tiene de vuelta el Nodo Built-in (deshabilitado para este caso, pero sigue siendo necesario para levantar los demás agentes, se levanta de forma efímera).

Decimonovena versión

Esta vez volvemos al problema con el comando “docker compose up – d” en el stage de deploy:

```
[Pipeline] sh
11:33:24  + tar -czf - --exclude=schema.sql --exclude=openapi.yaml --exclude=Dockerfile* --exclude=nada.txt -C app/damas .
11:33:24  + ssh -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 tar -xzf - -C /home/ubuntu/mi_proyecto_web/html/
11:33:26  + ssh -n -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 cd /home/ubuntu/mi_proyecto_web && docker compose up -d
11:33:28  unknown shorthand flag: 'd' in -d
11:33:28
11:33:28  Usage:  docker [OPTIONS] COMMAND [ARG...]
11:33:28
11:33:28  Run 'docker --help' for more information
[Pipeline] }
11:33:28  $ ssh-agent -k
11:33:28  unset SSH_AUTH_SOCK;
11:33:28  unset SSH_AGENT_PID;
11:33:28  echo Agent pid 2923 killed;
11:33:28  [ssh-agent] Stopped.
```

Cambio a “docker-compose up –d” que AWS2 sí reconoce. Además, se le añade “sudo” en el stage deploy debido a los permisos configurados en AWS2.

```
        # 2. Levantar los contenedores en producción
        ssh -n -o StrictHostKeyChecking=accept-new ubuntu@10.100.0.3 \
            "cd /home/ubuntu/mi_proyecto_web && sudo docker-compose up -d"
    '''
```

```
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ ls -l
total 12
-rw-rw-r-- 1 ubuntu ubuntu  403 May 17 00:56 Dockerfile
-rw-rw-r-- 1 ubuntu ubuntu  147 May 15 21:50 docker-compose.yml
drwxr-xr-x 2 ubuntu ubuntu 4096 May 17 14:25 html
```

Pipeline Success

Finalmente, después de 39 builds. La pipeline está funcionando correctamente de forma automatizada:

```
#39

11:44:10  + docker volume rm zap-data
11:44:10  + true
11:44:10  + rm -f Dockerfile.db openapi-runtime.yaml
11:44:10  + docker container prune -f
11:44:10  Total reclaimed space: 0B
11:44:10  + docker image prune -f
11:44:10  Total reclaimed space: 0B
11:44:10  + docker volume prune -f
11:44:10  Deleted Volumes:
11:44:10  2117d40aec26e642c9b37fdb8ac0ea8a73b0c79b0cf5b9900c999abf96f8a2e1
11:44:10
11:44:10  Total reclaimed space: 211.1MB
11:44:10  + docker network prune -f
11:44:10  + df -BG --output=avail /
11:44:10  + tail -1
11:44:10  + tr -dc 0-9
11:44:10  + DISK_AVAIL_GB=33
11:44:10  + echo Espacio libre en / tras cleanup: 33G
11:44:10  Espacio libre en / tras cleanup: 33G
11:44:10  + [ 33 -lt 3 ]
[Pipeline] }
[Pipeline] // node
[Pipeline] echo
11:44:10  Pipeline OK — Despliegue completado.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // timeout
[Pipeline] End of Pipeline
Finished: SUCCESS
```

